

Date: February 23rd, 2021
From: David G Tassoni
To: Dr. Kaputa
Subject: ESD II Lab 2 TIF Image Processing

Introduction

The purpose of this laboratory was to modify a given sample VHDL program to perform additional processing on TIF image files. The program by default that would read in TIF files, process them in multiple ways, and save off the processed TIF file. The required functionality to add was as follows:

- shall have 6 std_logic_vector(7 downto 0) inputs corresponding to min and max thresholds for red, green, blue.
- shall input red, green, and blue pixels.
- shall output modified red, green, and blue pixels based on the image thresholding.
- shall save off the resultant RGB filtered image.

The below analysis investigates the methods as to which the above requirements were met, as well as the overall functionality of the VHDL program.

Analysis

The lab source code consisted of six or seven VHDL files, and their functionality was part of the way to the end goal of the lab. The goal was to determine what needed to be changed or added to the existing **solution.vhd** and **filter.vhd** files to make them work as intended in the requirements. One other file, **image_handler.vhd**, contained a good number of procedures and processes, and studying this file, as well as the other two files, helped with completing the assignment. The first major change to make was adding in additional processing in the if/else statements; this added in green and blue pixel processing, as red was in by default. The image handler VHDL file had some code like this, so that was a helpful stepping point. Please see the image on the following page for a snippet of the pixel “grabbing” process.

```

process is
begin
    ---open the image
    ..open_tiff_image(input_filename, .image_data);
    ..wait_for(40 ns);

    ..for i in image_data'range loop
        ....wait_for(period);
        ....
    ..if (i mod 3 = 0) then
        ....--0 is red
        ....r <= conv_std_logic_vector(image_data(i), 8);
        ....image_data_proc(i) <= conv_integer(unsigned(r_out));
        ....
    ..elsif (i mod 3 = 1) then
        ....--1 is green
        ....g <= conv_std_logic_vector(image_data(i), 8);
        ....image_data_proc(i) <= conv_integer(unsigned(g_out));
        ....
    ..elsif (i mod 3 = 2) then
        ....--2 is blue
        ....b <= conv_std_logic_vector(image_data(i), 8);
        ....image_data_proc(i) <= conv_integer(unsigned(b_out));
        ....
    ..else
        ....--if not any of the above, set to 0 (black)
        ....image_data_proc(i) <= 0;
        ....end if;
    ..end loop;
    ..wait_for(40 ns);
    ....save and close the resultant RGB filtered image
    ..save_tiff_image_640x480(output_filename, .image_data_proc);
    ....set flag to 1 to signify operation is done
    ..rgb_done_flag <= '1';
    ..
    ..wait;
end process;

```

Figure 1 RGB pixel grabbing process

Once the pixels are “grabbed”, they are filtered in the filter VHDL file. That file checks to see if the values are between a certain threshold criterion (set in **solution.vhd**), and outputs them accordingly. The default values are set to 0 for the minimum, and 255 for the maximum, but they can be anything within that range to have different images output.

```

--Default values are 0 for min and 255 for max
signal r_min :: std_logic_vector(7 downto 0) := x"00"; --0
signal r_max :: std_logic_vector(7 downto 0) := x"FF"; --255
signal g_min :: std_logic_vector(7 downto 0) := x"00"; --0
signal g_max :: std_logic_vector(7 downto 0) := x"FF"; --255
signal b_min :: std_logic_vector(7 downto 0) := x"00"; --0
signal b_max :: std_logic_vector(7 downto 0) := x"FF"; --255
signal r_out :: std_logic_vector(7 downto 0);
signal g_out :: std_logic_vector(7 downto 0);
signal b_out :: std_logic_vector(7 downto 0);

```

Figure 2 RGB Minimum & Maximum assignments

```

32  --Signal Declarations
33  signal r_out_filtered::std_logic_vector(7.downto 0)::=(others=>'0');
34  signal g_out_filtered::std_logic_vector(7.downto 0)::=(others=>'0');
35  signal b_out_filtered::std_logic_vector(7.downto 0)::=(others=>'0');
36
37  begin
38
39  r_pixel_modify::process(r)
40  --output.modified.red.pixels.based.on.the.thresholding.values
41  ..begin
42  ....if(r.>=r_min).and.(r.<=r_max).then
43  ....|.r_out_filtered.<=r;
44  ....else
45  ....|.r_out_filtered.<= (others=>'0');
46  ....end.if;
47  end.process.r_pixel_modify;
48  r_out.<=r_out_filtered;
49
50  g_pixel_modify::process(g)
51  --output.modified.green.pixels.based.on.the.thresholding.values
52  ..begin
53  ....if(g.>=g_min).and.(g.<=g_max).then
54  ....|.g_out_filtered.<=g;
55  ....else
56  ....|.g_out_filtered.<= (others=>'0');
57  ....end.if;
58  end.process.g_pixel_modify;
59  g_out.<=g_out_filtered;
60
61  b_pixel_modify::process(b)
62  --output.modified.blue.pixels.based.on.the.thresholding.values
63  ..begin
64  ....if(b.>=b_min).and.(b.<=b_max).then
65  ....|.b_out_filtered.<=b;
66  ....else
67  ....|.b_out_filtered.<= (others=>'0');
68  ....end.if;
69  end.process.b_pixel_modify;
70  b_out.<=b_out_filtered;

```

Figure 3 RGB pixel modification in filter.vhd

Using ModelSim, the files can be simulated to produce an output. The output is a single image with its RGB pixels modified according to the assigned thresholds; there are two sample outputs, one with the default thresholds (of 0 and 255), and one with modified thresholds chosen at random.



Table 1 RGB pixel modification comparison

Conclusion

The program can correctly filter images based on the minimum and maximum RGB threshold values set. This was achieved by adding additional standard logic vector signals corresponding to the minimum and maximum thresholds for red, green, and blue, and adding to the existing pixel grabbing process. The pixels are input, modified, and output to a

resultant RGB filtered image which is saved in the Results folder. The given sample code was a great foundation, and the additional functionality was relatively simple to implement. In the end, the threshold values can be set to any viable number, and the simulation outputs a modified filtered image of any TIF file of dimensions 640x480 pixels.